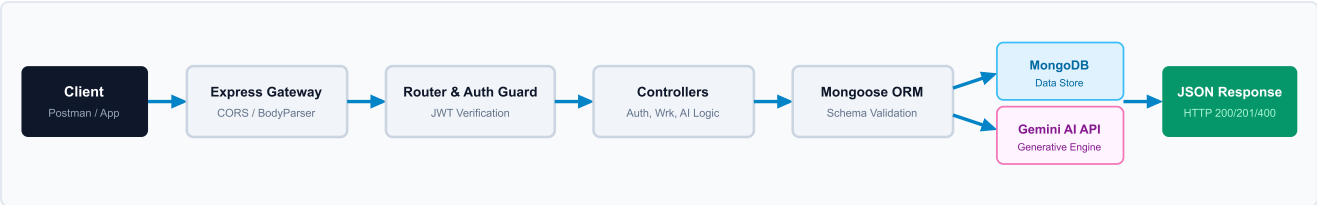


Strict Scope Boundaries & Sourcing Isolation: All data flows and story specifications are derived strictly from documented baseline entities. Future proposed extensions (Nutrition tracking, BMI calculation, Wearable device integration, Workout reminders, and Fitness analytics dashboards) are strictly excluded from baseline implementation.

1. END-TO-END ENTERPRISE DATA PROCESSING PIPELINE

The architectural data processing pipeline governs all incoming client requests traversing through Express gateway filters to data stores and external AI services.



2. STEP-BY-STEP FUNCTIONAL DATA FLOWS

FLOW CATEGORY	PIPELINE ENTRY & AUTH	PROCESSING SEQUENCE & LOGIC TRANSFORMATIONS	DATA STORE & EGRESS RESPONSE
Authentication Flow	POST /api/auth/register POST /api/auth/login Public access	1. Payload extracted (name, email, password). 2. Email checked against User collection. 3. Register hashes password via bcryptjs; Login compares hashes. 4. JWT generated using JWT_SECRET.	User document saved/retrieved from MongoDB. Returns HTTP 200/201 with signed JWT string and sanitized user details.
Workout CRUD Flow	/api/workouts Auth Guard middleware (Bearer <token>)	1. Token extracted, verified, decoded to bind req.user._id. 2. Controller applies operation (Create, Read, Update, Delete). 3. Mongoose schema validates data types and required attributes. 4. Query filters enforce userId === req.user._id.	Workout collection mutated in MongoDB. Returns HTTP 200 OK or 201 Created with JSON workout document(s).
Multi-criteria Search Flow	GET /api/workouts Query params: name, category, date	1. Controller extracts query string filters. 2. Dynamic filter object constructed combining user scope (userId) and query parameters. 3. MongoDB executes targeted indexed scan.	MongoDB returns matched subset. Controller returns HTTP 200 OK with filtered JSON array of workout records.
AI Regimen Flow	POST /api/ai/recommend Auth Guard required	1. Extracts goal and fitnessLevel from request body. 2. Constructs fitness prompt. 3. Dispatches HTTPS request to Google Gemini API with GEMINI_API_KEY.	External Google Gemini Generative Engine processes prompt. Controller returns HTTP 200 OK with generative recommendation.
AI Insights Flow	GET /api/ai/insights Auth Guard required	1. Controller queries MongoDB for authenticated user's logged workouts. 2. Formats workout history into analytical prompt context. 3. Dispatches prompt to Google Gemini API for pattern analysis.	Returns HTTP 200 OK with Gemini-generated analytical summary, consistency trends, and progress insights.
Error Handling Flow	All API endpoints & runtime middleware	1. Exceptions caught via controller try/catch or passed via next(error). 2. Central error middleware catches unhandled rejections and schema errors. 3. Maps error type to standardized response payload.	Returns standardized JSON error response: { message, error } with appropriate HTTP status code (400, 401, 404, 500).

3. CANONICAL USER STORIES & ACCEPTANCE CRITERIA

User stories representing core functional interactions for the single authenticated user role, with criteria derived from schemas and endpoints.

US-01: User Registration & Account Onboarding

AUTHENTICATION

"As an authenticated user, I want to register a personal account with my name, email, and password, so that I can securely log in and access fitness tracking capabilities."

- **AC-01.1:** Payload must supply valid name, email, and password; missing required fields must return HTTP 400 Bad Request.
- **AC-01.2:** System must reject duplicate emails with HTTP 400 Bad Request and descriptive message.
- **AC-01.3:** Passwords must be hashed using `bcryptjs` before database persistence; plain text storage is prohibited.
- **AC-01.4:** Successful creation returns HTTP 201 Created containing user profile and signed JSON Web Token.

US-02: User Authentication & Token Provisioning

AUTHENTICATION

"As an authenticated user, I want to authenticate with my registered email and password, so that I receive a JSON Web Token for authorized API access."

- **AC-02.1:** System must compare incoming password against persisted bcrypt hash; invalid credentials return HTTP 400 or 401.
- **AC-02.2:** Upon verification, system must generate and return a signed JWT token carrying user identifier context.
- **AC-02.3:** Profile access via `GET /api/auth/profile` requires Bearer token and returns user details excluding hashed password.

US-03: Workout Record Logging

WORKOUT CRUD

"As an authenticated user, I want to log a workout with its name, category, duration, calories burned, and date, so that my activity history is accurately tracked."

- **AC-03.1:** Request must be authenticated via Bearer token; unauthorized calls return HTTP 401 Unauthorized.
- **AC-03.2:** Schema validation enforces required fields (name, duration numeric); validation failures return HTTP 400.
- **AC-03.3:** Saved document must automatically reference authenticated user's ID (user Id). Returns HTTP 201 Created.

US-04: Workout History Retrieval & Traversal

WORKOUT CRUD

"As an authenticated user, I want to retrieve all my logged workouts or inspect a single workout by its identifier, so that I can monitor my fitness activities."

- **AC-04.1:** `GET /api/workouts` must return only workouts belonging to the authenticated user.
- **AC-04.2:** `GET /api/workouts/:id` returns HTTP 200 OK with single workout object matching the ObjectId.
- **AC-04.3:** Attempting to query an ID belonging to another user must return HTTP 404 Not Found, enforcing tenant isolation.

US-05: Workout Modification & Deletion

WORKOUT CRUD

"As an authenticated user, I want to update or delete my logged workouts, so that my fitness journal remains accurate and up to date."

- **AC-05.1:** `PUT /api/workouts/:id` updates targeted workout fields and returns HTTP 200 OK with modified document.
- **AC-05.2:** `DELETE /api/workouts/:id` removes the record from MongoDB and returns HTTP 200 OK confirmation.
- **AC-05.3:** Operations against non-existent IDs or records belonging to other users must be rejected.

4. CANONICAL USER STORIES (CONT. - AI CAPABILITIES)

US-06: AI-Powered Workout Plan Recommendation

AI INTELLIGENCE

"As an authenticated user, I want to submit my fitness goal and current level to receive personalized AI workout recommendations from Google Gemini."

- AC-06.1:** Endpoint POST /api/ai/recommend requires Bearer JWT authentication.
- AC-06.2:** Request body must supply goal and fitnessLevel parameters.
- AC-06.3:** System dispatches structured prompt to Google Gemini API and returns synthesized recommendation with HTTP 200 OK.
- AC-06.4:** Upstream AI failures must be caught gracefully and return sanitized HTTP 500 error responses via central error handling.

US-07: AI Fitness Progress & Performance Insights

AI INTELLIGENCE

"As an authenticated user, I want to generate AI-driven performance insights from my historical workout records, so that I can understand my fitness trends."

- AC-07.1:** Endpoint GET /api/ai/insights requires valid Bearer JWT authentication.
- AC-07.2:** Controller aggregates historical workout documents for the authenticated user from MongoDB.
- AC-07.3:** Workout history is contextualized into an analysis prompt sent to Google Gemini API.
- AC-07.4:** Returns HTTP 200 OK containing Gemini-generated insights, trends, and progression feedback.

5. USER-TO-SYSTEM TRACEABILITY MATRIX

Traceability mapping connecting user stories to architectural layers, database models, controller logic, and API endpoints.

STORY ID	FEATURE FOCUS	API ENDPOINT & METHOD	CONTROLLER MODULE	DATA MODEL	SECURITY SHIELD
US-01	User Registration	POST /api/auth/register	authController.register	User Model	bcrypt Password Hash
US-02	Login & Profile	POST /api/auth/login GET /api/auth/profile	authController.login authController.getProfile	User Model	JWT / Auth Middleware
US-03	Log Workout	POST /api/workouts	workoutController.createWorkout	Workout Model	Bearer JWT / Schema Val
US-04	Get Workouts / ID	GET /api/workouts GET /api/workouts/:id	workoutController.getWorkouts workoutController.getWorkoutById	Workout Model	Bearer JWT / Tenant Scope
US-05	Update & Delete	PUT /api/workouts/:id DELETE /api/workouts/:id	workoutController.updateWorkout workoutController.deleteWorkout	Workout Model	Bearer JWT / Record Ownership
US-06	AI Recommendation	POST /api/ai/recommend	aiController.getRecommendation	N/A (External API)	Bearer JWT / Gemini Key
US-07	AI Fitness Insights	GET /api/ai/insights	aiController.getInsights	Workout Model	Bearer JWT / User Scope